

Parfait ! Voici le scénario de test complet dans l'ordre logique.

Prérequis

```
# S'assurer que tout tourne
docker-compose up -d

# Ouvrir le Swagger
http://localhost:8000/api/docs
```

🟢 Étape 1 — Health Check

```
GET /health
```

✅ Attendu :

```
{ "status": "ok", "service": "BP Monitor" }
```

🔒 Étape 2 — Authentification

2.1 — Register un nouveau patient

```
POST /api/v1/auth/register
```

```
{
  "username": "test.patient",
  "email": "test.patient@email.com",
  "password": "secret123",
  "first_name": "Test",
  "last_name": "Patient",
  "phone_number": "+22890000099",
  "organisation_code": "HOPITAL_LOME"
}
```

✅ Attendu : `access_token` + `refresh_token`

2.2 — Login avec email

```
POST /api/v1/auth/login
```

```
{ "identifiant": "admin@bpmonitor.com", "password": "secret" }
```

✓ Attendu : tokens JWT

2.3 — Login avec username

```
{ "identifiant": "admin", "password": "secret" }
```

✓ Attendu : tokens JWT

2.4 — Login avec téléphone

```
{ "identifiant": "+22890000001", "password": "secret" }
```

✓ Attendu : tokens JWT

2.5 — Autoriser le Swagger 🔒

- Copiez le `access_token` du login admin
 - Cliquez **Authorize** en haut du Swagger
 - Collez : `Bearer <votre_token>`
-



Étape 3 — Profil utilisateur

3.1 — Me

```
GET /api/v1/auth/me
```

✓ Attendu :

```
{  
  "id": 1,  
  "email": "admin@bpmonitor.com",  
  "roles": ["admin"],
```

```
"organisation_id": 1
}
```



Étape 4 — Organisations

4.1 — Lister les organisations

```
GET /api/v1/organisations
```

✓ Attendu : liste avec **HOPITAL_LOME**, **CLINIQUE_BIASA**

4.2 — Créer une organisation

```
POST /api/v1/organisations
```

```
{
  "nom": "Clinique du Port",
  "code": "CLINIQUE_PORT",
  "adresse": "Lomé, Togo",
  "telephone": "+228 22 33 44 55",
  "email": "contact@cliniqueport.tg"
}
```

✓ Attendu : organisation créée



Étape 5 — Médecins & Patients

5.1 — Lister les médecins

```
GET /api/v1/patients/medecins
```

✓ Attendu : liste avec **Dr Kofi Mensah**

5.2 — Obtenir le profil patient

```
GET /api/v1/patients/1
```

✓ Attendu : profil du patient

5.3 — Mettre à jour le profil patient

```
PATCH /api/v1/patients/1
```

```
{
  "gender": "F",
  "birth_date": "1990-05-15",
  "blood_group": "A+",
  "address": "Lomé, Togo",
  "emergency_contact": "+22890000000"
}
```

✓ Attendu : profil mis à jour

💖 Étape 6 — Invitation médecin → patient

6.1 — Login médecin

```
POST /api/v1/auth/login
```

```
{ "identifiant": "kofi.mensah@bpmonitor.com", "password": "secret" }
```

Autorisez avec le token du médecin.

6.2 — Médecin génère un code

```
POST /api/v1/patients/invitation/generer
```

✓ Attendu :

```
{
  "code": "ABC12345",
  "expire_le": "2026-...",
  "est_utilise": false
}
```

Notez le **code**.

6.3 — Patient accepte l'invitation

Reconnectez-vous avec le token patient, puis :

```
POST /api/v1/patients/1/invitation/accepter
```

```
{ "code": "ABC12345" }
```

✅ Attendu :

```
{
  "message": "Vous êtes maintenant suivi par Dr Kofi Mensah.",
  "medecin_id": 2,
  "patient_id": 1
}
```

6.4 — Patient choisit directement un médecin

```
POST /api/v1/patients/1/choisir-medecin
```

```
{ "medecin_id": 2 }
```

✅ Attendu : confirmation du lien



Étape 7 — Mesures de tension

7.1 — Créer une mesure normale

```
POST /api/v1/mesures/
```

```
{
  "patient_id": 1,
  "systolique": 120,
  "diastolique": 80,
  "pouls": 70,
  "periode": "matin",
  "jour": 1,
  "numero_mesure": 1,
  "session_id": "session-test-001",
}
```

```
"notes": "Après repos"
}
```

✅ Attendu : `categorie: "normale"`

7.2 — Créer une mesure élevée

```
{
  "patient_id": 1,
  "systolique": 135,
  "diastolique": 87,
  "pouls": 80,
  "periode": "matin",
  "jour": 1,
  "numero_mesure": 2,
  "session_id": "session-test-001"
}
```

✅ Attendu : `categorie: "elevee"`

7.3 — Créer une mesure critique 🚨

```
{
  "patient_id": 1,
  "systolique": 185,
  "diastolique": 115,
  "pouls": 95,
  "periode": "soir",
  "jour": 1,
  "numero_mesure": 1,
  "session_id": "session-test-001"
}
```

✅ Attendu :

- `categorie: "critique"`
- Alerte créée en base
- SMS envoyé au patient et au médecin 📱

7.4 — Lister les mesures du patient

```
GET /api/v1/mesures/patient/1
```

✅ Attendu : liste des 3 mesures

7.5 — Obtenir le résumé de session

```
GET /api/v1/mesures/resume/1/session-test-001
```

✅ Attendu :

```
{
  "nombre_mesures": 3,
  "session_complete": false,
  "progression": "3/18 mesures",
  "moyenne_globale": {
    "systolique": 147,
    "diastolique": 94,
    "categorie": "hypertension"
  }
}
```

Étape 8 — Alertes

8.1 — Déclencher une alerte

```
POST /api/v1/alertes/1/declencher
```

✅ Attendu : alerte avec statut: "envoyee"

8.2 — Acquitter une alerte

```
PATCH /api/v1/alertes/1/acquitter
```

```
{ "acquittee_par": "Dr Kofi Mensah" }
```

✅ Attendu : alerte avec statut: "acquittee"

✅ Vérification finale en base

```
# Mesures créées
docker-compose exec db psql -U bp_user -d bp_monitor \
  -c "SELECT id, systolique, diastolique, categorie FROM mesures;"
```

```
# Alertes créées
docker-compose exec db psql -U bp_user -d bp_monitor \
  -c "SELECT id, niveau, statut, acquittee_par FROM alertes;"

# Patient lié au médecin
docker-compose exec db psql -U bp_user -d bp_monitor \
  -c "SELECT id, user_id, medecin_id, organisation_id FROM patients;"

# Invitations utilisées
docker-compose exec db psql -U bp_user -d bp_monitor \
  -c "SELECT id, code, est_utilise, medecin_id, patient_id FROM invitations;"
```

Faites les tests dans l'ordre et dites-moi ce que vous voyez à chaque étape. 🚀